المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵉⵏⵎⵍⴰⵍ - ⴱⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

Rapport des travaux pratiques

Architecture Monolithique, Monolithe Modulaire, Architecture Distribuée et Architecture SOA

Filière : Intelligence Artificielle et Cybersécurité

Module :
Architecture Logicielle

Réalisé par :

Abderrahman MACHHOURI

Encadré par :
Pr. BE EL BAGHAZAOUI

Année universitaire : 2025-2026

Table des matières

Liste des figures **2**

Liste des tableaux **3**

1 TP2 : Monolithe Modulaire **4**

1.1 Objectifs 4
1.2 Structure du TP 4
1.3 Lien vers Github 5
1.4 Diagramme des Dépendances 5
1.5 Tests avec Postman des trois modules 5
1.5.1 Module Product 5
1.5.2 Module Customer 7
1.5.3 Module Ordre 9
1.6 Vérification des dépendances 12
1.6.1 Création d'une commande avec un client inexistant 12
1.6.2 Création d'une commande avec un produit inexistant 12
1.7 Conclusion Globale 13

Table des figures

1.1 Structure du projet modulaire 4
1.2 POST - Créer un produit (Module Product) 5
1.3 GET - Lister tous les produits (Module Product) 6
1.4 GET - Récupérer un produit par ID (Module Product) 6
1.5 PUT - Modifier un produit (Module Product) 7
1.6 DELETE - Supprimer un produit (Module Product) 7
1.7 POST - Créer un client (Module Customer) 8
1.8 GET - Lister tous les clients (Module Customer) 8
1.9 GET - Récupérer un client par ID (Module Customer) 9
1.10 POST - Créer une commande (Module Order) 9
1.11 GET - Lister toutes les commandes (Module Order) 10
1.12 GET - Détails d'une commande par ID (Module Order) 10
1.13 GET - Historique des commandes d'un client 11
1.14 DELETE - Supprimer une commande (Module Order) 11
1.15 Erreur lors de la création d'une commande avec un client inexistant 12
1.16 Erreur lors de la création d'une commande avec un produit inexistant . . . 13

Liste des tableaux

TP2 : Monolithe Modulaire

1.1 Objectifs

- Restructurer le projet en modules métier
- Introduire les DTO
- Utiliser MapStruct
- Préparer une future migration microservices

1.2 Structure du TP

Le projet est divisé en trois modules :

- Module Product
- Module Customer
- Module Order

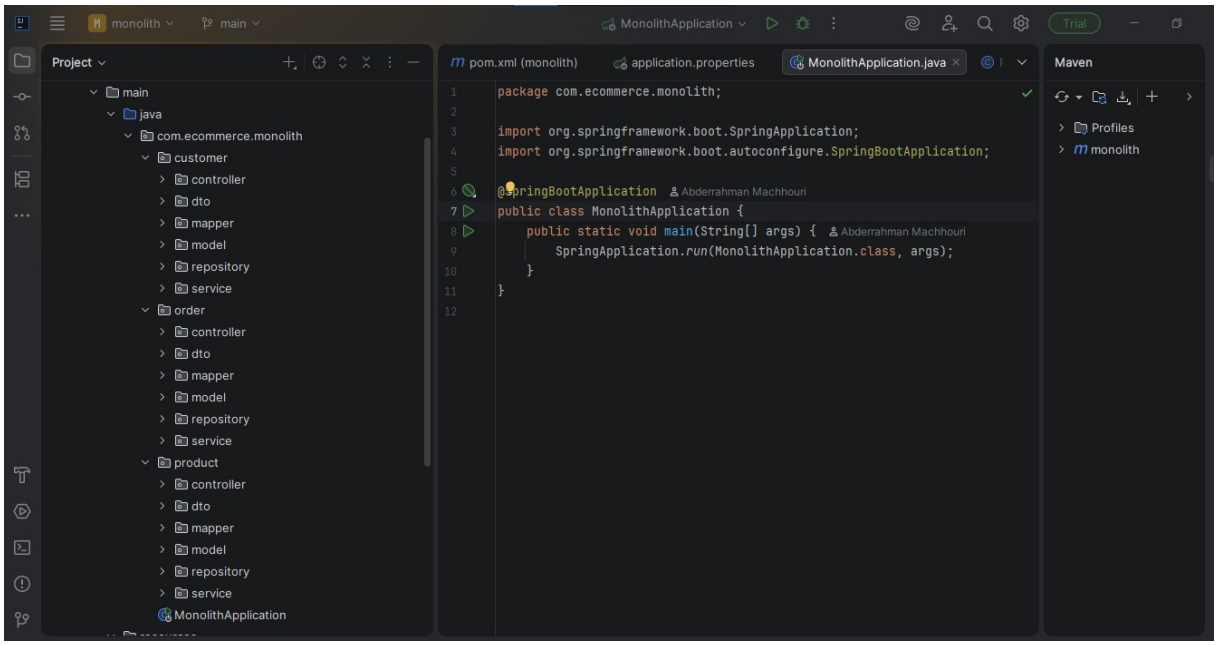

FIGURE 1.1 – Structure du projet modulaire

1.3 Lien vers Github

Donc chaque module contient six packages.
Pour voir le code source de tous les modules et de l'application, voici l'url correspondant : https://github.com/abderrahmanmachhouri/TPs_Genie_Logicielle/tree/main/TP2_monolithe_modulaire/monolith

1.4 Diagramme des Dépendances

— Product : indépendant
— Customer : indépendant
— Order : dépend de Product et Customer

1.5 Tests avec Postman des trois modules

1.5.1 Module Product

— La méthode POST pour créer un produit :

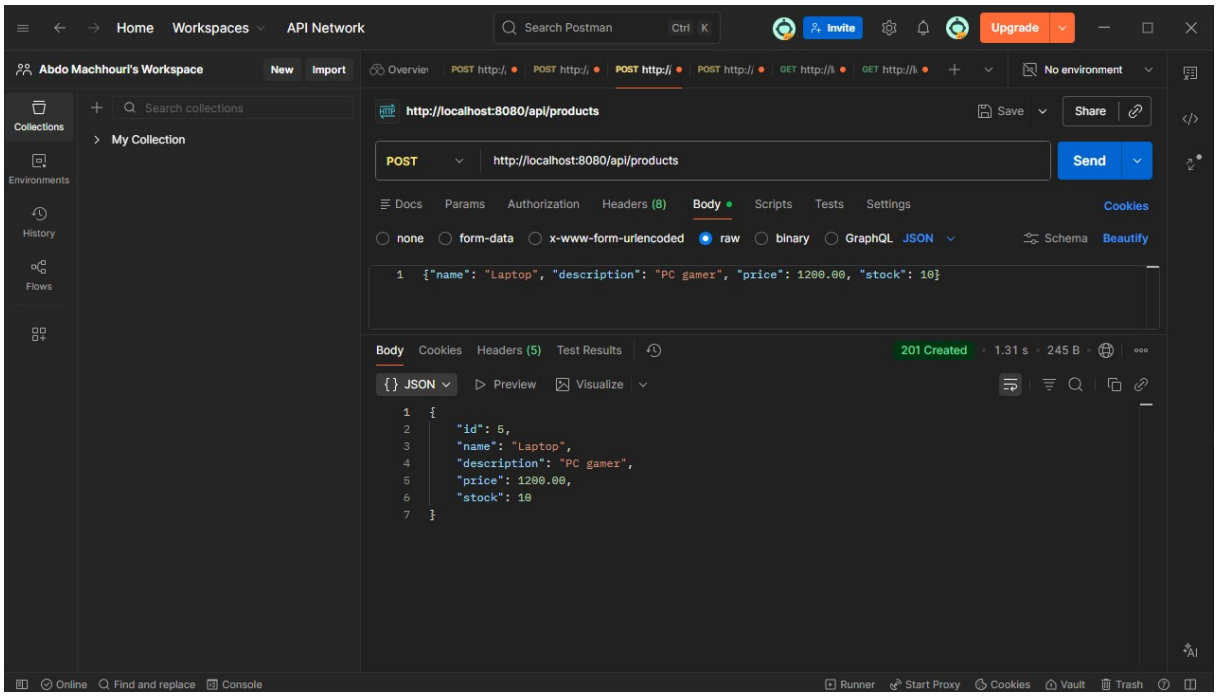

FIGURE 1.2 – POST - Créer un produit (Module Product)

— La méthode GET pour récupérer tous les produits :

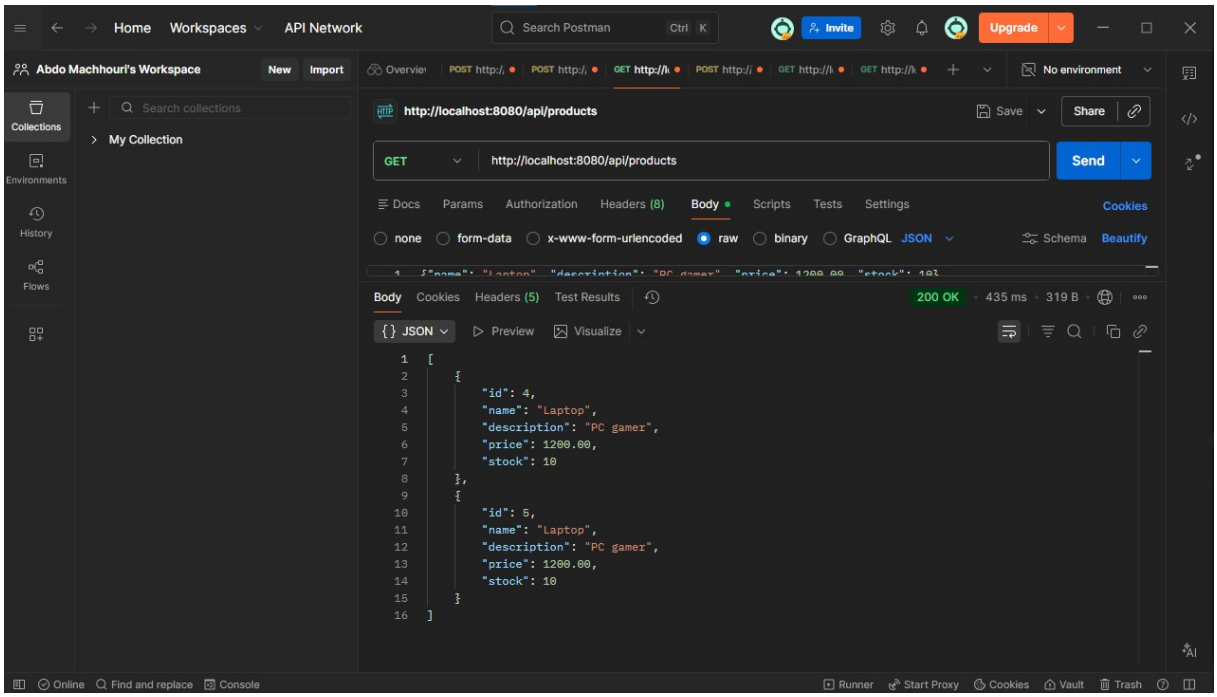

FIGURE 1.3 – GET - Lister tous les produits (Module Product)

— La méthode GET par identifiant :

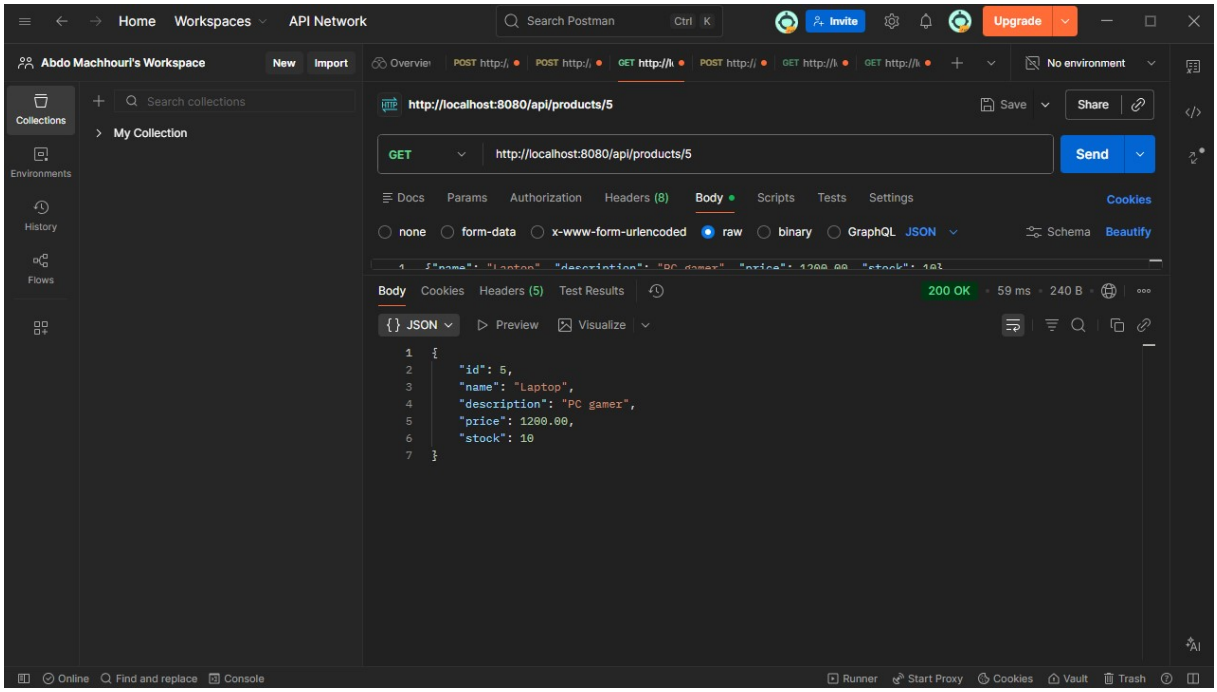

FIGURE 1.4 – GET - Récupérer un produit par ID (Module Product)

— La méthode PUT pour modifier un produit :

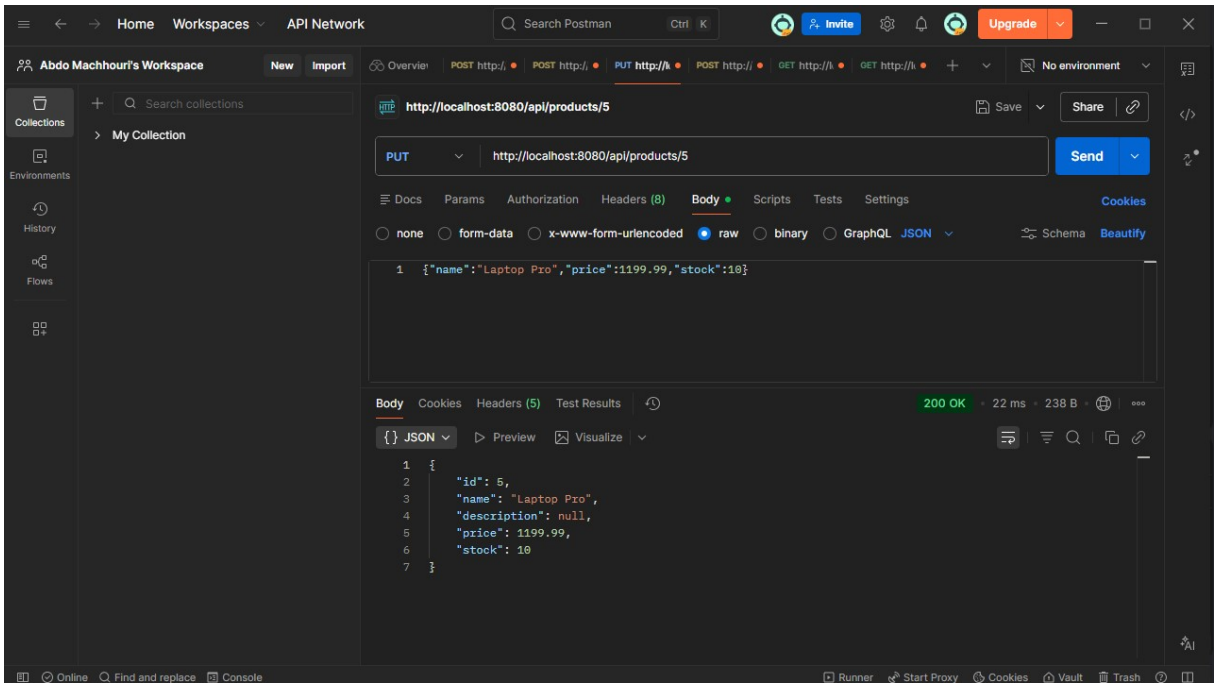

FIGURE 1.5 – PUT - Modifier un produit (Module Product)

— La méthode DELETE pour supprimer un produit :

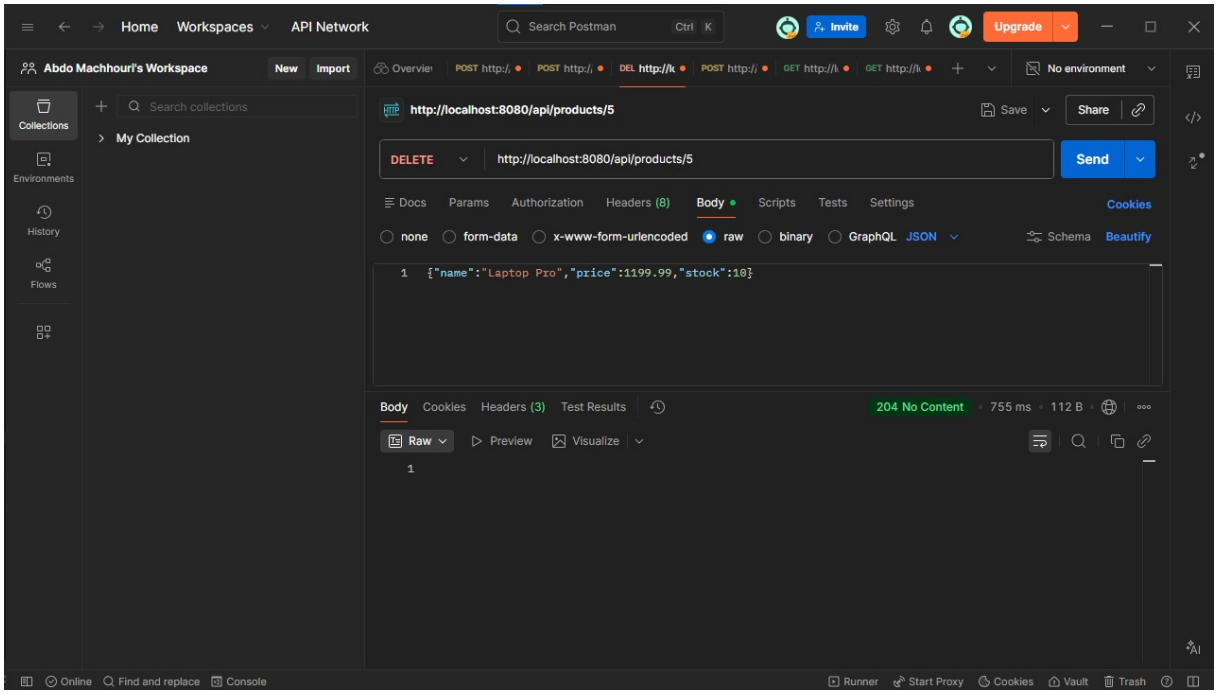

FIGURE 1.6 – DELETE - Supprimer un produit (Module Product)

1.5.2 Module Customer

— La méthode POST pour créer un client :

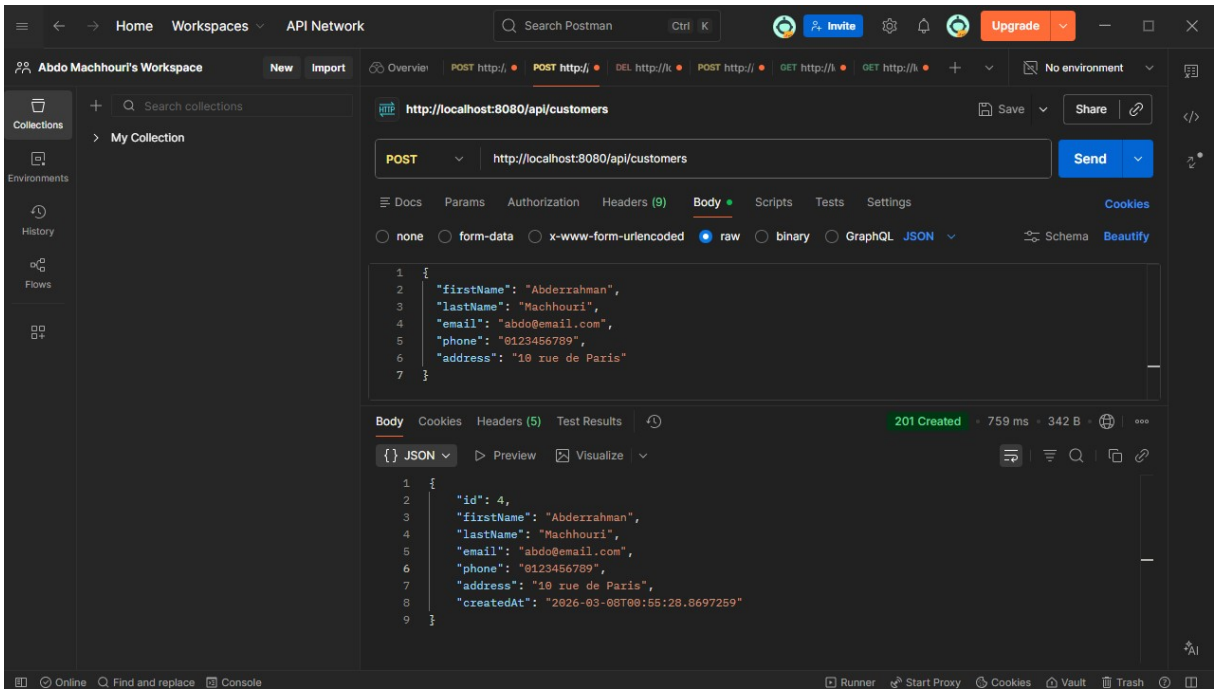

FIGURE 1.7 – POST - Créer un client (Module Customer)

— La méthode GET pour récupérer tous les clients :

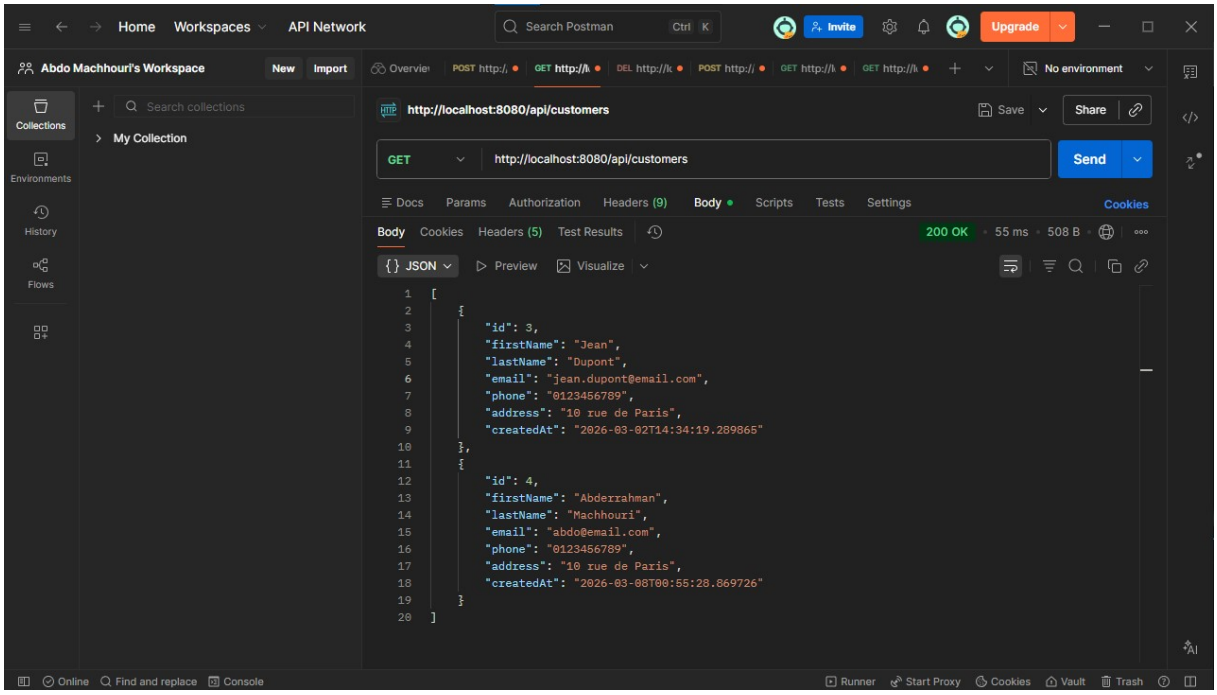

FIGURE 1.8 – GET - Lister tous les clients (Module Customer)

— La méthode GET pour récupérer un client en détails :

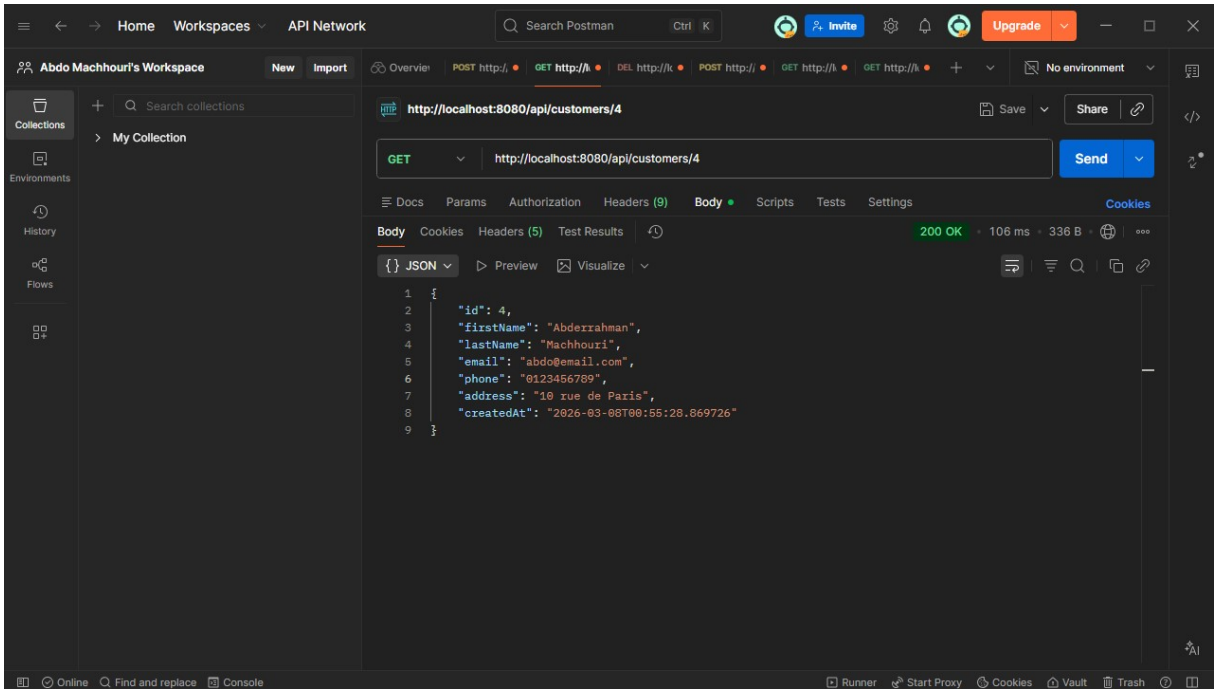

FIGURE 1.9 – GET - Récupérer un client par ID (Module Customer)

1.5.3 Module Ordre

— La méthode POST pour créer une commande :

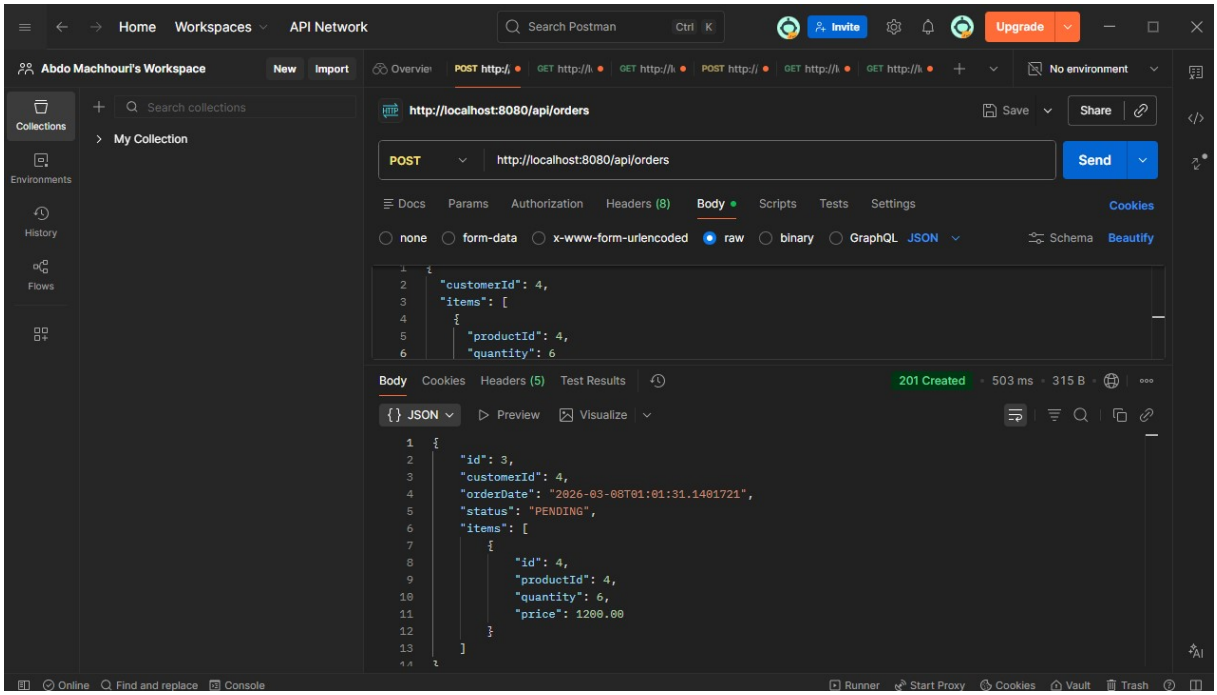

FIGURE 1.10 – POST - Créer une commande (Module Order)

— La méthode GET pour lister les commandes :

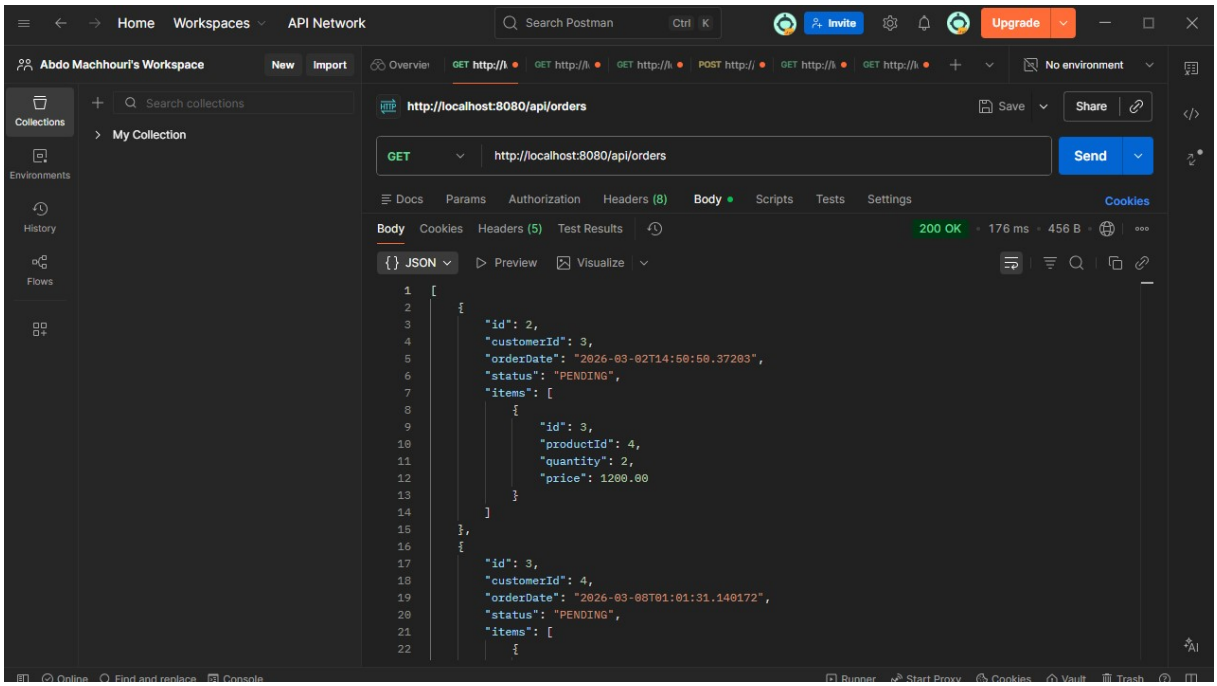

FIGURE 1.11 – GET - Lister toutes les commandes (Module Order)

— La méthode GET pour les détails d'une commande :

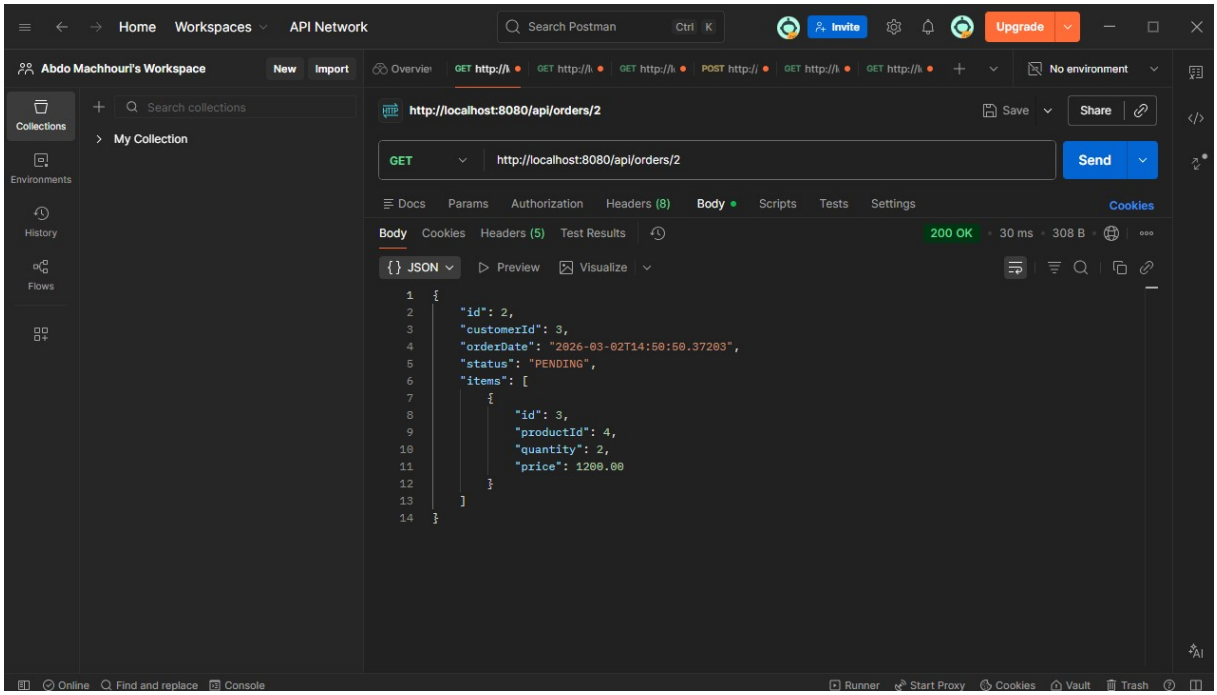

FIGURE 1.12 – GET - Détails d'une commande par ID (Module Order)

— La méthode GET pour l'historique des commandes d'un client :

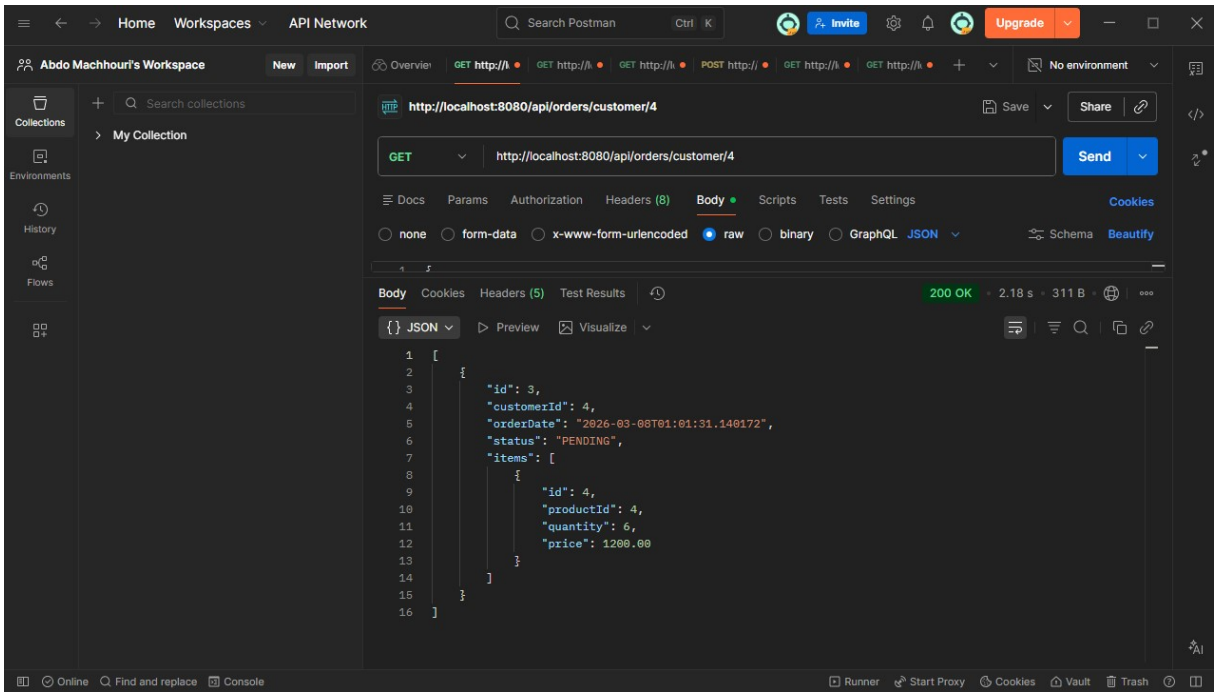

FIGURE 1.13 – GET - Historique des commandes d'un client

— La méthode DELETE pour supprimer une commande :

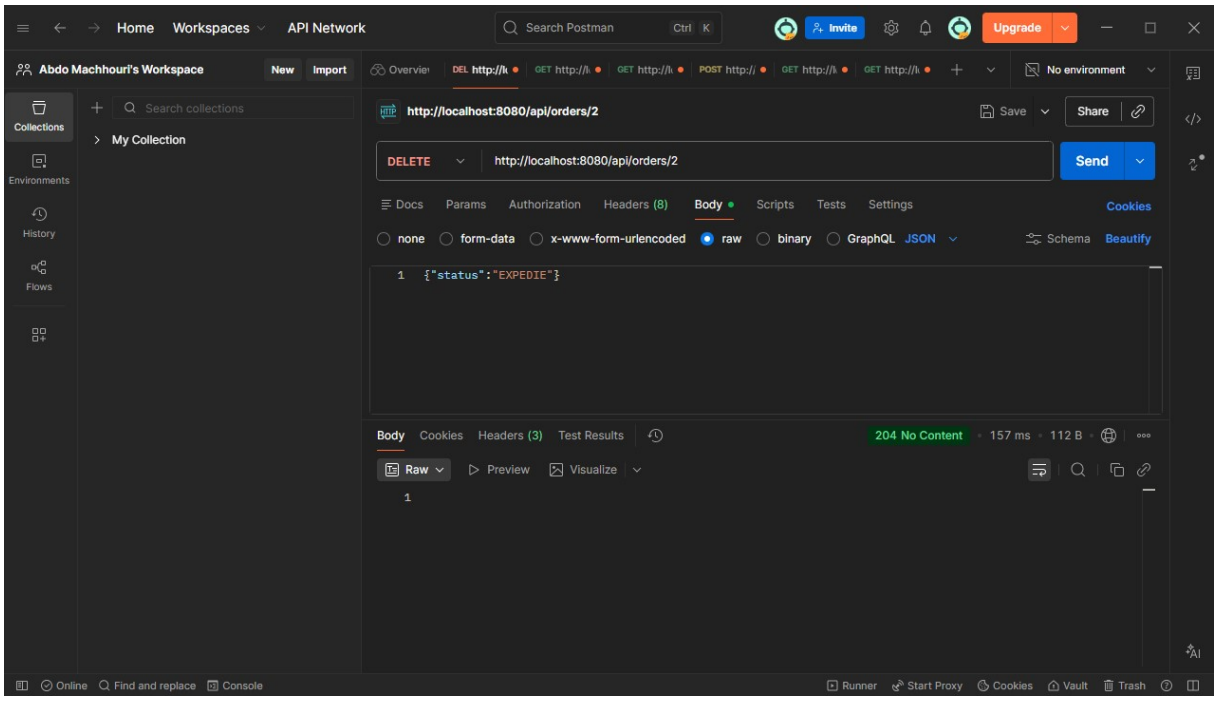

FIGURE 1.14 – DELETE - Supprimer une commande (Module Order)

1.6 Vérification des dépendances

1.6.1 Création d'une commande avec un client inexistant

On va créer une commande avec un client inexistant et il doit retourner 404 Not Found ou 400 Bad Request avec un message d'erreur explicite.

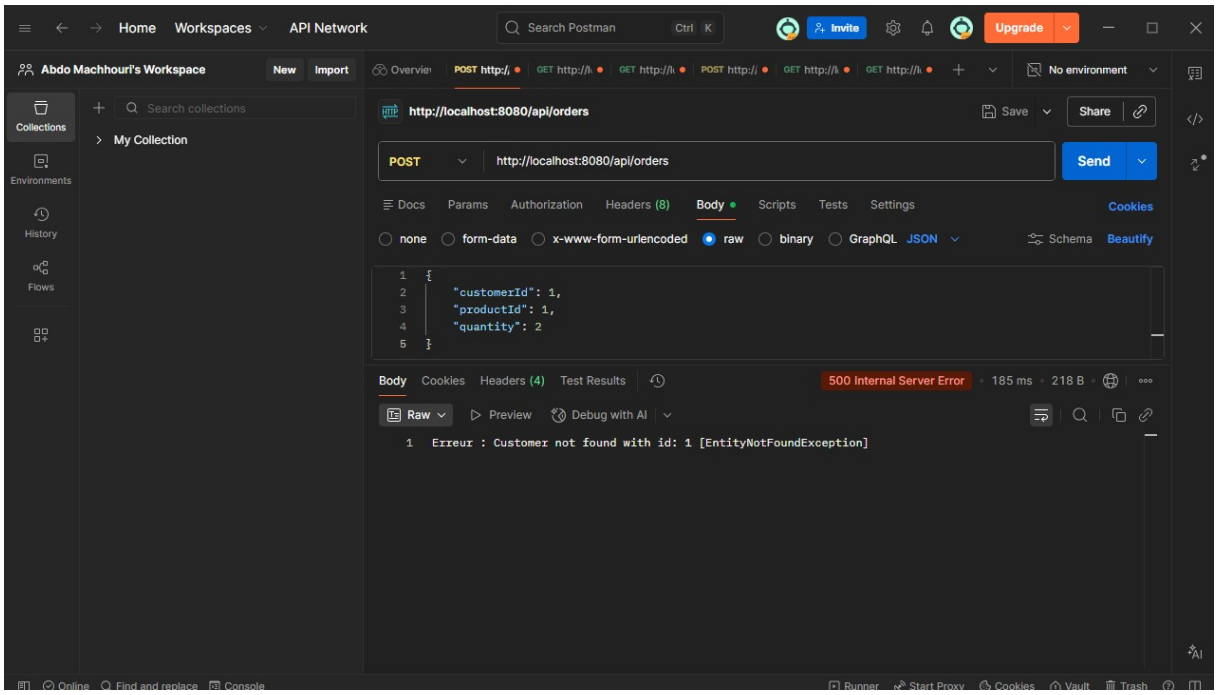

FIGURE 1.15 – Erreur lors de la création d'une commande avec un client inexistant

1.6.2 Création d'une commande avec un produit inexistant

La même chose, on va créer une commande avec un produit inexistant et on attend qu'il retourne une erreur.

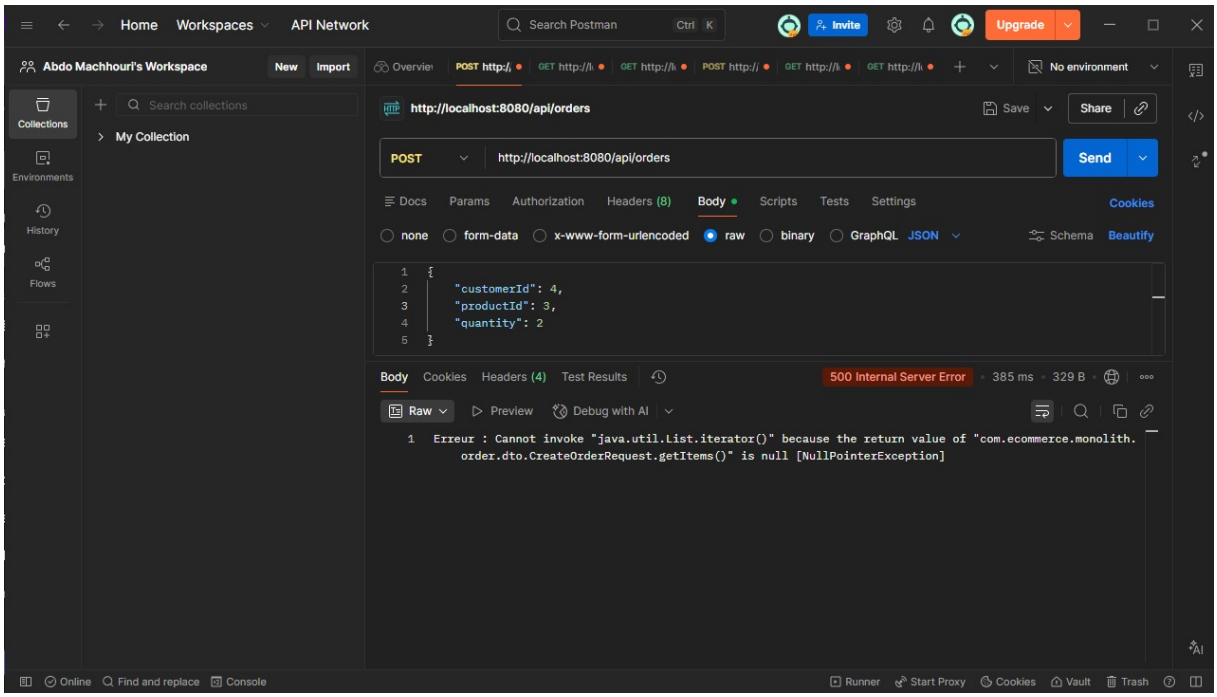

FIGURE 1.16 – Erreur lors de la création d'une commande avec un produit inexistant

1.7 Conclusion Globale

Ce deuxième TP a permis de faire évoluer l'application monolithique initiale vers une architecture modulaire, mieux structurée et plus proche des principes métier. En introduisant des modules distincts pour la gestion des produits, des clients et des commandes, nous avons appliqué une séparation claire des responsabilités. L'utilisation de DTOs et de mappers (avec MapStruct) a renforcé l'indépendance entre la couche de persistance et la couche de présentation, tandis que le recours à des interfaces de service a limité les dépendances directes entre modules. Les tests fonctionnels menés avec Postman ont validé le bon fonctionnement des endpoints, y compris les vérifications d'intégrité référentielle (existence d'un client ou d'un produit avant création d'une commande) et l'accès à l'historique des commandes d'un client. Cette restructuration constitue une étape préparatoire essentielle à une éventuelle migration vers une architecture microservices, en posant des bases modulaires, faiblement couplées et facilement extractibles.